

TYPES OF MICRO SERVICES



@java_in.depth

ALL 8 TYPES AT A GLANCE

API Gateway

Single entry point for all clients

TRAFFIC LAYER

Config Service

Centralized config across all services

INFRA

Business Service

Core domain logic (Orders, Payments)

DOMAIN CORE

Notification Svc

Email, SMS, Push alerts

COMMS

Data Service

Owens its DB, exposes data via API

PERSISTENCE

Event Service

Async messaging via Kafka/RabbitMQ

ASYNC/EVENT

Auth Service

Identity, JWT, OAuth2 management

SECURITY

Aggregator Svc

Collects data from multiple services

COMPOSITION

Every Java dev working with distributed systems must know these. Swipe through each type — what it does, what you need to know.

API GATEWAY SERVICE

THE FRONT DOOR OF YOUR ENTIRE SYSTEM

What it does:

Single entry point that routes incoming requests to the correct microservice. Handles cross-cutting concerns so individual services don't have to.

Key Responsibilities:

- Request routing + load balancing
- Auth token validation before forwarding
- Rate limiting & throttling per client
- SSL termination & request logging

2026 tip: Use Spring Cloud Gateway over Zuul — Zuul is in maintenance mode. Add circuit breaker via Resilience4j at gateway level.

Java Stack:

- Spring Cloud
- Netflix Zuul
- Kong
- AWS API Gateway

BUSINESS SERVICE

WHERE YOUR ACTUAL PRODUCT LOGIC LIVES

What it does:

Encapsulates one bounded domain — Order Service, Payment Service, Inventory Service. Each owns its own data and logic. No shared databases.

Key Responsibilities:

- Handle business rules & validations
- Expose REST / gRPC APIs
- Publish domain events to message broker
- Own its own database — no DB sharing

Design rule: If two services share a DB table, they're not real microservices. Use Domain-Driven Design to define boundaries.

Java Stack:

- Spring Boot
- Spring Data JPA
- PostgreSQL / MySQL
- Hibernate

DATA SERVICE

ONE SERVICE, ONE DATABASE — ALWAYS

What it does:

Dedicated service that abstracts data storage. Other services call it via API rather than hitting the DB directly. Enforces data ownership boundaries.

Key Responsibilities:

- CRUD operations on owned data
- Caching layer (Redis) for hot data
- DB migrations via Liquibase / Flyway
- Polyglot persistence — right DB per use case

Pattern: CQRS — separate read and write models. Use Redis for reads, PostgreSQL for writes. Big performance win at scale.

Java Stack:

- Spring Data
- Redis
- SQLElasticsearch
- MongoDB
- Postgre

EVENT SERVICE

DECOUPLE EVERYTHING WITH ASYNC MESSAGING

What it does:

Produces and/or consumes events on a message broker. Services communicate without knowing about each other. Essential for resilient, scalable systems.

Key Responsibilities:

- Publish domain events (OrderPlaced, PaymentDone)
- Subscribe and react to events from other services
- Handle retries, dead-letter queues (DLQ)
- Implement Saga pattern for distributed transactions

2026: Kafka + Outbox Pattern is the standard. Write event to DB atomically, then publish — no lost messages on crash.

Java Stack:

- Apache Kafka
- Spring Kafka
- RabbitMQ
- AWS SQS/SNS

Comment “java” to get pdf link

AUTH & IDENTITY SVC

WHO ARE YOU? WHAT CAN YOU DO?

What it does:

Handles authentication (who you are) and authorization (what you can access). Issues and validates JWT tokens. All other services trust its tokens.

Key Responsibilities:

- Login, register, logout flows
- JWT access & refresh token lifecycle
- Role-based access control (RBAC)
- OAuth2 / SSO for third-party login

Best practice: Don't build auth from scratch. Use Keycloak or Spring Authorization Server. Stateless JWT at API Gateway level.

Java Stack:

- Spring Security
- Keycloak
- JWT / OAuth2
- Spring Auth Server

Comment "java" to get pdf link

AGGREGATOR SERVICE

"ONE CALL, DATA FROM MANY SERVICES"

What it does:

Calls multiple downstream services, combines their responses into a single payload for the client. Also called Backend-for-Frontend (BFF) pattern.

Key Responsibilities:

- Parallel calls to multiple services
- Combine & transform responses for client needs
- Handle partial failures gracefully (fallbacks)
- BFF pattern — mobile vs web get different shapes

Performance: Use WebFlux + parallel calls instead of sequential. 3 services $\times$ 100ms = 100ms, not 300ms.

Java Stack:

- Spring Security
- Keycloak
- JWT / OAuth2
- Spring Auth Server

Comment "java" to get pdf link

NOTIFICATION SERVICE

COMMUNICATIONS

What it does:

Sends email, SMS, push notifications. Listens to events from other services — never called directly. Fully decoupled via message queue.

Key Responsibilities:

- Consume events → trigger right notification channel
- Retry logic + template management

Java Stack:

- JavaMail
- Twilio
- Firebase FCM
- SendGrid

Comment “java” to get pdf link

CONFIG SERVICE

INFRASTRUCTURE

What it does:

Centralized config store for all microservices. No hardcoded properties. One place to change DB URLs, feature flags, timeouts across all services.

Java Stack:

- Spring Cloud Config
- HashiCorp Vault
- Git-backed config

Comment “java” to get pdf link

HOW SERVICES TALK TO EACH OTHER

// communication patterns in 202

Synchronous — REST / gRPC:

Direct call, waits for response. Use when you need an immediate answer. gRPC is faster than REST for internal service-to-service calls.

→ *Use: Feign Client, WebClient, gRPC stub*

Asynchronous — Events / Queues:

Fire and forget. Producer doesn't wait. Better resilience & decoupling. Standard for notifications, billing, audit logs.

→ *Use: Kafka, RabbitMQ, Spring Events*

Service Mesh (2026):

Handles retries, timeouts, observability at infra level. Services talk normally — mesh handles the rest. Growing fast in k8s environments.

→ *Use: Istio, Linkerd, AWS App Mesh*

NOW YOU KNOW THE FULL MAP

Save this post — it's your microservices reference sheet for system design interviews and real projects. Follow for more Java architecture content.

Comment “java” to get pdf link